REQUIREMENT MANAGER + GITHUB COPILOT

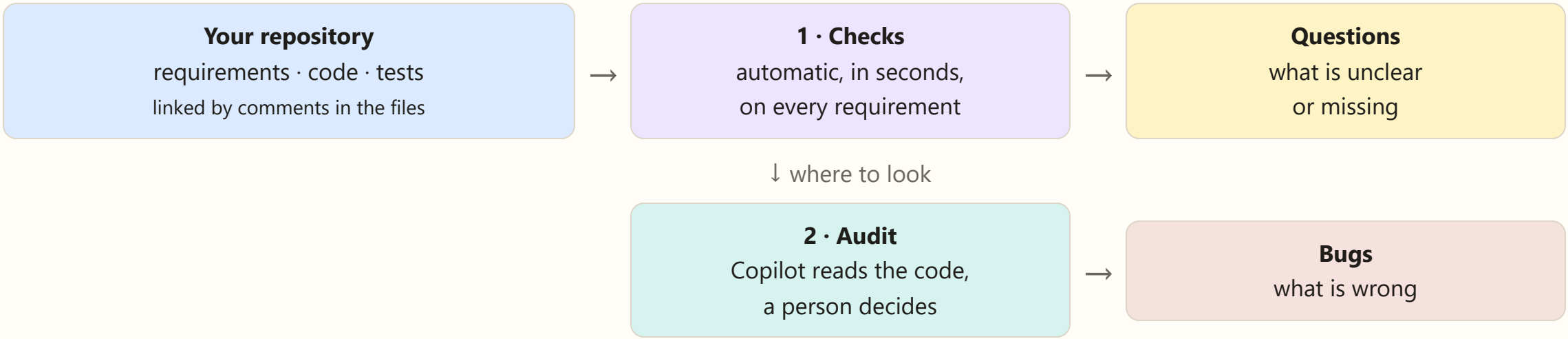
Finding problems in requirements, code and tests

How a small tool and Copilot check that the code does what the requirements say, explained with two examples from a car.

Every tool output on these slides is from a real run. Where Copilot's answer is shown, the slide says it is the expected answer.

THE IDEA IN ONE PICTURE

Three things you already have, two things you get



Requirement

A short text that says what the software must do, for example “the brake light turns on above 5 %”.

Tag

A comment in the code that says which requirement it belongs to, for example `// implements: ID`.

Case

An example with an input and the expected result, for example “20 % → light on”. Each one should have a test.

In plain words: the tool checks the links between the three, Copilot reads the code where the tool points, and a person decides what is really wrong.

THE COMMANDS

Each command answers one question

COMMAND	THE QUESTION IT ANSWERS	CHANGES FILES?
Every day: the checks		
`gate`	Is everything linked? Did an agreed requirement change?	no
`gate --risk`	What is missing? Which cases have no test?	no
`clarify ID`	What in this requirement is unclear?	no
`gate --show ID`	Show me everything about one requirement.	no
Now and then: the audit next release		
`sync --audit-plan`	Which requirements do we review? (chosen once, written down)	yes, one file
`gate --audit`	What did the review find, and how sure are we?	no
Plumbing		
`sync`	Save the agreed state and redraw the map.	yes, in `requirements/`
`mcp`	Let Copilot ask the questions above.	no

In plain words: almost every command only reads. The two that write touch only the tool's own files.

WHO READS WHAT

What the tool reads, and what Copilot reads

Requirement Manager

Reads: the requirement files, the tags in code and tests, and the saved state from last time.

Does: checks that every link points somewhere real, that every requirement has code and tests, and that nothing agreed has changed in secret.

Does not: run the tests, or judge whether the code is correct.

Copilot

Reads: one requirement at a time, as the tool hands it over, plus the files the tool names:

```
{"id": "EXL-BRAKELIGHT-001",
 "contract": ["... returns true when the pedal
              position is above 5 % ..."],
 "members": [{"role": "implements",
              "file": "src/brake_light.cpp"},
             {"role": "tested-by",
              "file": "tests/test_brake_light.cpp"}]}
```

Does: compares each sentence of the requirement with the code and the test, and says what looks wrong.

Does not: decide, or edit requirements.

> **In plain words:** the tool is the map, Copilot reads the places the map points to, a person decides.

DATA AND PRIVACY

Nothing leaves the repository through the tool

- ✓ **No internet.** The tool contains no network code at all: no HTTP, no sockets, no telemetry, no account, no licence server. It runs offline.
- ✓ **Git only on your machine.** It reads history with local git commands and never runs `fetch`, `pull`, `push` or `clone`.
- ✓ **Writes only inside the repository:** its own files in `requirements/`, and its config files once, when it is set up (`init`).
- ✓ **Copilot talks to the tool through a local pipe** (MCP over stdio): no network port is opened.

the one exception **Copilot**

What Copilot reads goes to the Copilot service, under the organisation's existing Copilot agreement, exactly as when you use Copilot for anything else. In agent mode it can open any file of the project, so check the company's Copilot settings (content exclusion) before the pilot.

In plain words: use the tool without Copilot, and nothing leaves the computer. Use it with Copilot, and only Copilot's usual traffic leaves.

COPILOT SETUP

Three steps, and Copilot can only read

1 · Connect the tool

Add `.vscode/mcp.json` (the tool writes it for you):

```
{"servers": {"reqmap": {
  "type": "stdio", "command": "python",
  "args": ["${workspaceFolder}/scripts/reqmap.py",
    "mcp", "--root", "${workspaceFolder}"]}}}
```

2 · Keep it read-only

Do **not** add `--allow-writes`. Copilot then gets 11 tools that only read: `reqmap_show`, `reqmap_clarify`, `reqmap_review`, `reqmap_audit`…

The two that write (`reqmap_sync`, `reqmap_release`) do not exist for it.

3 · Give it the rules

In `.github/copilot-instructions.md`:

- never edit a requirement;
- for each sentence: which code line does it, which test proves it;
- one label per finding: bug, not-implemented, false-coverage, ok;
- check each requirement twice, in two separate chats.

In plain words: Copilot can ask the tool anything, but cannot change anything through it. The rules keep it from editing files on its own.

The situation

The requirement (agreed and saved)

```
  - `brakeLightOn(pedalPercent)` returns true when the
    pedal position is above 5 %, and false otherwise.
+ - A pedal reading below 0 %, above 100 % or not a
+   number is a sensor fault, and the light turns on.


CASE-1  20 % → on     CASE-2  0 % → off
CASE-3   5 % → off
```

In plain words: press the pedal, the light turns on. **New rule** (green): if the pedal sensor is broken, the light turns on too, to be safe.

The code and the tests (not changed)

```
// implements: EXL-BRAKELIGHT-001
bool brakeLightOn(float pedalPercent) {
    return pedalPercent > 5.0f;
}

// tested-by: EXL-BRAKELIGHT-001
TEST(BrakeLight, PressedPedalTurnsLightOn)    // CASE-1
TEST(BrakeLight, ReleasedPedalKeepsLightOff)  // CASE-2
TEST(BrakeLight, ThresholdIsOff)              // CASE-3
```

The code was written before the new rule. All three tests still pass (by reading the code; not executed).

In plain words: the requirement moved on, the code did not, and nothing turns red. This is the kind of problem we are looking for.

What the tool finds by itself

```
$ reqmap.py gate
WARN  RM018 EXL-BRAKELIGHT-001: DRIFT — contract
      changed since lock; re-check 3 member(s):
      src/brake_light.cpp:1, src/brake_light.h:1,
      tests/test_brake_light.cpp:1

$ reqmap.py clarify EXL-BRAKELIGHT-001
  [no-failure-case · wording] Cases
  What happens on the failure path -- missing
  input, invalid value, nothing to do?
```

RM018 DRIFT means: the agreed text of this requirement changed after it was saved.

re-check 3 member(s) means: these are the three files linked to it. Look at them again.

no-failure-case means: none of the examples describes what happens when something goes wrong, such as a broken sensor.

Real output, copied from the run.

In plain words: the tool says “this changed, check these 3 files, and you have no example for a failure”. It does not say whether the code is wrong.

What Copilot finds, and what a person decides

Copilot reads the requirement and the 3 files, and tries the values each sentence talks about:

PEDAL READING	SHOULD BE	CODE GIVES
20 %	on	on ✓
−3 % (broken sensor)	on	off ✗
“not a number” (broken)	on	off ✗
150 % (broken sensor)	on	on, by luck

Why “on” for a broken sensor? A real pedal moves between 0 % and 100 %. A reading of −3 %, 150 % or “not a number” can only come from a broken sensor, so nobody knows whether the driver is braking. The requirement picks the safe side: light on, so the car behind is warned.

Expected answer, derived from the code; not a recorded run.

A person reads Copilot's answer and decides:

- real bug with a broken sensor the brake light stays off. Fix: check for a broken reading first.
- false coverage the three green tests never try a broken sensor. Fix: add a test with −3, 150 and “not a number”.
- ambiguity after a fault, does the light stay on until the engine is switched off? The requirement does not say. The owner decides.

In plain words: Copilot finds the bug by reading the code; a person confirms it and decides what to change.

Summary: what example 1 found

PROBLEM	TYPE	FOUND BY	WHY IT MATTERS
The requirement changed, the code did not	drift	tool	nobody would have been told
No example for a broken sensor	gap	tool	the case code most often forgets
Broken sensor: the light stays off	bug	Copilot + a person	safety: the car behind is not warned
Green tests that never try a fault	false coverage	Copilot + a person	green does not mean safe
How long does the light stay on?	ambiguity	a person	two answers, two different programs

After the fix the tool still asked for a failure example, because it does not know the word “fault” yet. A person marks it as a false alarm .

In plain words: five findings from one small requirement: two from the tool alone, three only by reading the code.

The situation

The requirement

```
- `autoLock(speedKmh, locked)` returns true when the
  speed is above 15 km/h and the doors are not locked.
- `crashUnlock()` unlocks every door quickly after a
  crash is detected.

CASE-1  20 km/h, unlocked → lock
CASE-2  10 km/h, unlocked → do not lock
CASE-3  a crash → every door unlocked
```

In plain words: lock the doors when driving faster than 15 km/h; unlock them after a crash so rescuers can get in.

The code and the tests

```
// implements: EXL-DOORLOCK-001      (door_lock.cpp)
return speedKmh >= 15.0f && !locked;

// implements: EXL-DOORLCK-001       (crash_unlock.cpp)
for (int d = 0; d < 4; ++d) unlockDoor(d);

// tested-by: EXL-DOORLOCK-001       (test_door_lock.cpp)
TEST(DoorLock, DrivingAwayLocks)          // CASE-1
TEST(DoorLock, StandingStillDoesNotLock)  // CASE-2
```

Four small things to spot

① a letter missing in one tag (`DOORLCK`) · ② `>=` where the text says “above” · ③ the word “quickly” · ④ CASE-3 has no test.

What the tool finds by itself

```
$ reqmap.py gate
ERROR RM001 dangling tag: code references
      EXL-DOORLCK-001 but no requirement exists
gate: FAIL

$ reqmap.py gate --risk
Cases without a test (1)
  EXL-DOORLOCK-001  1/3 case(s) carry no
  `# verifies:` tag

$ reqmap.py clarify EXL-DOORLOCK-001
  [vague-term · wording] clause 2
  What is the measurable threshold behind
  "quickly"?
```

① **RM001** means: a file says it belongs to a requirement that does not exist. The id is mistyped, so `crash_unlock.cpp` is linked to nothing.

④ **1/3 cases have no test** means: one example, the crash, is never tested.

③ **vague-term “quickly”** means: how quickly? The requirement gives no number of milliseconds.

Real output, copied from the run.

In plain words: three of the four problems, found in seconds, without anyone reading the code.

The audit: Copilot reads, a person decides

Step 1 · choose what to review, once: `sync --audit-plan`

Step 2 · Copilot checks the requirement twice, in two chats. Both times it says:

- ② sentence 1: the doors lock at **exactly** 15 km/h, but the text says **above** 15 → bug
- sentence 2: no code found, because the only file has the mistyped tag → not implemented

Step 3 · a person decides: sentence 1 is a real bug; sentence 2 is first of all unclear (“quickly”), so the owner must answer.

Step 4 · the result: `gate --audit`

```
counters   B 1 · F_copilot 0 · G 0 · A 1 ·
           F_tool 0 · M 0 · U 0 · ok 0
agreement  2/2  (95% 0.34-1.00)
```

- **B 1**: one real bug
- **A 1**: one unclear requirement
- **agreement 2/2**: both Copilot runs said the same thing

The report is real output; the findings were typed in by hand for this example.

In plain words: Copilot found the one thing only reading the code shows, the `>=`; the person turned two opinions into two decisions.

Summary: what example 2 found

PROBLEM	TYPE	FOUND BY	WHY IT MATTERS
① A tag points to a mistyped id	broken link	tool	the crash code is invisible to every check
④ The crash case has no test	gap	tool	the most important case is never tried
③ “Quickly” has no number	ambiguity	tool asks, owner answers	nobody can test “quickly”
② Doors lock at exactly 15 km/h	bug	Copilot + a person	the code does something the text does not allow

In plain words: here the tool finds most of it alone; Copilot adds the one-character bug that only reading the code reveals.

IN SHORT

The kinds of problems, and who finds them

KIND OF PROBLEM	IN THE EXAMPLES	WHO FINDS IT
broken link a tag to an id that does not exist	the mistyped door-lock tag	the tool, on every commit
drift the requirement changed, the code did not	the new brake-light rule	the tool, on every commit
gap an example with no test, no failure example	the crash, the broken sensor	the tool
ambiguity a vague word, a missing decision	“quickly”, how long the light stays on	the tool asks, a person decides
bug the code does something other than the text	broken sensor, `>=`	Copilot proposes, a person confirms
false coverage green tests that prove nothing	the brake-light tests	Copilot proposes, a person confirms

In plain words: the tool finds the structural problems by itself, on every requirement, in seconds. Copilot reads the code where the tool points. A person decides every bug.